AI ENGINEERING HANDBOOK

Distributed Training

Scaling Model Training Across GPUs, Machines & Clusters

Chapter

Data Parallelism, Model Parallelism, Hybrid Strategies & the Ray Ecosystem

AI Engineering Handbook — 2025 Edition

Table of Contents

Introduction: Why the Rules Changed for AI Training

For most of computing history, training a machine learning model was a problem you could solve by waiting long enough on a single machine. Early neural networks had thousands of parameters, training datasets had millions of rows, and a powerful workstation could complete a training run overnight. The fundamental assumption was simple: one machine, one model, one training loop.

That assumption collapsed around 2017–2020 as a perfect storm of factors converged. Research demonstrated that model capability scales predictably with the number of parameters — and that the most capable models have hundreds of billions, or even trillions, of parameters. GPT-3 has 175 billion parameters; PaLM has 540 billion; the largest frontier models exceed one trillion. These numbers are not just large in an abstract sense — they are physically incompatible with any single piece of computing hardware that exists or is likely to exist within this decade.

At the same time, the datasets required to train these models grew to match. BERT was pretrained on 3.3 billion words. GPT-3 was trained on 499 billion tokens. Modern vision-language models train on billions of image-text pairs. No single storage system can efficiently serve this data to a single GPU without creating catastrophic I/O bottlenecks.

Chapter Scope

This chapter covers the complete landscape of distributed AI training: the computational physics that make distributed training necessary, the three primary parallelism strategies (data, model, hybrid), the engineering challenges that arise when distributing workloads across hardware, and the Ray ecosystem — the dominant Python-native framework for building scalable distributed AI systems. Section 1 covers theory; Section 2 provides a detailed practical walkthrough of every code cell.

The transition from single-machine to distributed training is not merely an engineering optimization — it is a categorical shift in how AI systems are conceived, built, and operated. Understanding this transition thoroughly is foundational for any engineer working in modern AI infrastructure.

Section 1

Theory & Conceptual Foundation

The physics of scale, parallelism strategies, and the architecture of distributed AI training systems

Chapter 1

The Rise of Big AI

How model scale became the defining challenge of modern ML

1.1 The Scaling Laws That Changed Everything

The intellectual foundation for the era of big AI is a set of empirical observations known as scaling laws — relationships between model size, dataset size, compute budget, and model capability that were rigorously documented by researchers at OpenAI and DeepMind between 2020 and 2022.

The core finding was striking in its simplicity and implications: model performance on language tasks improves predictably and smoothly as a power-law function of model size, dataset size, and compute — with no observed ceiling within ranges that had been experimentally tested. The practical message for the research community was unambiguous: to build more capable AI systems, build larger ones.

Model	Organization	Parameters	Training Tokens	Year
BERT-Large	Google	340 Million	3.3 Billion	2018
GPT-2	OpenAI	1.5 Billion	40 Billion	2019
GPT-3	OpenAI	175 Billion	499 Billion	2020
PaLM	Google	540 Billion	780 Billion	2022
GPT-4 (est.)	OpenAI	~1.8 Trillion	~13 Trillion	2023
Llama 3.1	Meta	405 Billion	15 Trillion	2024

Each row in this table represents not just a larger model, but a fundamentally different engineering problem. GPT-2 can be trained on a single high-end GPU. GPT-3 required thousands of A100 GPUs running in parallel for months. The training compute for GPT-4 is estimated to have exceeded 100 million GPU-hours — a number that makes single-machine training not just slow, but physically impossible.

1.2 The Computational Physics of Single-Machine Limits

To understand why distributed training is necessary, you first need to understand exactly what limits a single machine. There are three fundamental constraints — each independently sufficient to make large-scale AI training impossible on a single node.

1.2.1 Memory (VRAM) Constraints

Every parameter in a neural network must reside in GPU memory (VRAM) during training — not just the parameter value itself, but also the gradient (the update computed during backpropagation) and the optimizer state (for Adam: first and second moment estimates). At full FP32 precision, each parameter therefore requires approximately 16 bytes of VRAM:

- 4 bytes — parameter value (FP32)
- 4 bytes — gradient (FP32)
- 4 bytes — first moment (Adam optimizer, FP32)
- 4 bytes — second moment (Adam optimizer, FP32)

For a 175-billion-parameter model at FP32, this means approximately 2.8 terabytes of VRAM — while the most powerful single GPU available today (the NVIDIA H100) has 80 gigabytes. The model literally cannot fit on any single GPU that exists today, let alone one available when GPT-3 was trained.

Model Scale	Min. VRAM Required (Training)	H100 GPUs Required (80GB each)
1 Billion parameters	~16 GB	1 GPU (fits in one)
7 Billion parameters	~112 GB	2 GPUs minimum
70 Billion parameters	~1.12 TB	14 GPUs minimum
175 Billion parameters	~2.8 TB	35 GPUs minimum
540 Billion parameters	~8.64 TB	108 GPUs minimum

1.2.2 Compute Throughput Constraints

Even if a model could somehow fit into a single GPU's memory, training throughput would be severely limited. Modern GPU architectures like the NVIDIA H100 have a theoretical peak throughput of approximately 2,000 TFLOPS (tera floating-point operations per second). Training a 175B parameter model on one trillion tokens requires roughly 10^23 floating-point operations — which would take a single H100 approximately 1,585 years at theoretical peak throughput.

Distributing this computation across 10,000 H100 GPUs reduces the wall-clock time to something approaching a practical training run (weeks rather than millennia) — but only if the distribution overhead is managed carefully.

1.2.3 Data Throughput Constraints

The third constraint is data I/O bandwidth. Training on trillions of tokens requires reading and processing data at rates that can saturate even the fastest NVMe storage arrays. Distributed training systems solve this by sharding data across many storage nodes, each serving a portion of the dataset to a portion of the training GPUs — achieving aggregate I/O bandwidth that no single storage device could provide.

> ⚠ **The Single-Machine Training Era Is Over**
>
> Any engineer approaching large-scale AI training with the expectation that 'a bigger single machine will eventually solve this' is operating under a category error. The constraints are not temporary engineering limitations — they are consequences of basic physics. GPU VRAM cannot grow at the rate model sizes are growing. The solution is not a larger single machine but a distributed system of many machines working in coordinated parallel. Distributed training is the permanent, structural answer to the scaling problem.

Chapter 2

Data Parallelism

Scaling training throughput by distributing data across identical model replicas

2.1 Core Concept and Mental Model

Data parallelism is the simplest and most widely used distributed training strategy. The central insight is straightforward: if training one batch of data on one GPU takes T seconds, and you want to process N batches simultaneously, you can use N GPUs — each running an identical copy of the model and processing a different batch of training data.

Definition

Data parallelism is a distributed training strategy in which the full model is replicated identically across multiple devices (GPUs or machines), while the training dataset is partitioned into non-overlapping subsets. Each device processes its own data subset independently in each training step. After each forward and backward pass, devices synchronize their gradient updates to ensure all model replicas remain identical.

2.2 How Data Parallelism Works — Step by Step

Step 1 — Model Replication: Copy identical model weights to every GPU in the training cluster

▼

Step 2 — Data Sharding: Partition training dataset into N equal subsets (one per GPU)

▼

Step 3 — Parallel Forward Pass: Each GPU runs forward pass on its own data mini-batch simultaneously

▼

Step 4 — Loss Computation: Each GPU computes loss on its own predictions vs. ground truth labels

▼

Step 5 — Backward Pass: Each GPU computes gradients for its own mini-batch independently

▼

Step 6 — Gradient Synchronization: All-Reduce operation averages gradients across all GPUs

▼

Step 7 — Weight Update: Each GPU applies the averaged gradient to update its local model copy

The key insight is steps 1 and 6: the model is identical on every GPU at all times (maintained by step 6), but each GPU processes different data in each step (step 2). This means all GPUs make progress on the model simultaneously, but from different angles of the data distribution — achieving N× the throughput of a single GPU while (theoretically) maintaining the same model quality.

2.3 The Gradient Synchronization Mechanism — All-Reduce

The most technically sophisticated component of data parallelism is gradient synchronization — the process of combining gradients computed independently across all GPUs into a single, averaged gradient update that is then applied identically to all model replicas.

The dominant algorithm for this synchronization is All-Reduce — a collective communication operation in which every participating device both contributes its local gradient values and receives the globally averaged result. Two primary All-Reduce implementations are used in practice:

Algorithm	Mechanism	Best For	Latency Profile
Ring All-Reduce	Devices form a ring; gradients flow around the ring in two phases	Large clusters, bandwidth-efficient	O(N) latency, optimal bandwidth
Tree All-Reduce	Hierarchical tree reduction from leaves to root and back	Small clusters, low latency	O(log N) latency, higher bandwidth use
NCCL All-Reduce	NVIDIA's optimized collective ops library, adapts to topology	NVLink-connected GPU clusters	Near-theoretical hardware limits

The communication overhead of gradient synchronization is the primary performance challenge in data parallelism. Every training step requires transmitting gigabytes of gradient data across the interconnect between GPUs (NVLink within a node, InfiniBand or RoCE between nodes). If the model has 175 billion parameters at FP32, a single gradient synchronization transmits approximately 700GB of data — a significant network bottleneck in large clusters.

2.4 When to Use Data Parallelism

Condition	Data Parallelism Appropriate?	Reasoning
Model fits in single GPU VRAM	☑ Yes — ideal	No need to split the model; simplest approach
Dataset is too large for fast training	☑ Yes — ideal	Dataset sharding directly solves this problem
Model exceeds single GPU VRAM	✗ No — insufficient	Must use model parallelism or hybrid approaches
Need maximum throughput increase	☑ Yes — linear scaling	Adding N GPUs provides ~N× throughput (minus overhead)
Gradient communication is bottleneck	⚠ Caution	Optimize with gradient compression or async updates

Practical Insight

Data parallelism is almost always the right first choice when scaling a training workload. If your model fits comfortably on a single GPU, start with data parallelism — it is the simplest to implement (PyTorch's DistributedDataParallel wraps any model in a few lines of code) and provides near-linear throughput scaling up to hundreds of GPUs. Only when the model itself exceeds GPU memory capacity do you need to introduce the additional complexity of model parallelism.

2.5 Effective Batch Size and Learning Rate Implications

Data parallelism has an important side effect on training dynamics: when you use N GPUs, each processing a mini-batch of size B, the effective global batch size becomes N × B. This is not merely a throughput metric — it changes the gradient statistics and can affect model convergence.

The linear scaling rule (empirically established by researchers at Facebook AI Research in 2017) states: when increasing the batch size by a factor of k, increase the learning rate by the same factor k. Without this adjustment, training with large effective batch sizes produces worse-converging models because the gradient estimates become too smooth, reducing the beneficial noise that helps models escape sharp minima.

This means that data parallelism is not a free lunch from a training dynamics perspective — it requires careful hyperparameter management that increases in complexity with the number of GPUs. Production distributed training systems like PyTorch FSDP and DeepSpeed implement learning rate warmup and scaling strategies automatically.

Chapter 3

Model Parallelism

Distributing a single model across multiple devices when it exceeds GPU memory

3.1 Core Concept and Motivation

Model parallelism is the strategy of partitioning the model itself — rather than the data — across multiple devices. Where data parallelism places a complete copy of the model on every device, model parallelism places different parts of the model on different devices. Each device owns only a fraction of the total model parameters, enabling the training of models whose aggregate parameter count far exceeds any single device's memory.

Definition

Model parallelism is a distributed training strategy in which the neural network's parameters are partitioned across multiple devices. No single device contains the full model; instead, each device holds a distinct subset of the model's layers, attention heads, or parameter matrices. Inference and training require coordination between devices to pass activations and gradients through the distributed model graph.

3.2 The Two Flavors of Model Parallelism

3.2.1 Pipeline Parallelism (Layer-Wise Partitioning)

Pipeline parallelism divides a neural network's layers into sequential groups (stages), with each stage assigned to a different device. For a 96-layer transformer:

- GPU 0 — handles layers 1–24 (embedding + first 24 transformer blocks)
- GPU 1 — handles layers 25–48 (next 24 transformer blocks)
- GPU 2 — handles layers 49–72 (next 24 transformer blocks)
- GPU 3 — handles layers 73–96 (final transformer blocks + output head)

Training proceeds like a manufacturing pipeline: a mini-batch enters GPU 0, gets processed through layers 1–24, and the resulting activations are transferred to GPU 1 for processing through layers 25–48, continuing down the pipeline. Multiple micro-batches can be in-flight simultaneously (GPU 0 processing micro-batch 2 while GPU 1 processes micro-batch 1), improving hardware utilization.

⚠ Pipeline Bubble Problem

Naive pipeline parallelism suffers from a 'pipeline bubble' — periods where GPUs upstream in the pipeline are idle waiting for downstream GPUs to finish so that the backward pass can begin. With K

pipeline stages, up to (K-1)/K of compute time can be wasted in the bubble. Advanced scheduling strategies like GPipe's micro-batch scheduling and interleaved pipeline schedules reduce (but cannot eliminate) this overhead. For a 4-stage pipeline, the theoretical bubble overhead is 75% without optimization.

3.2.2 Tensor Parallelism (Intra-Layer Partitioning)

Tensor parallelism takes a finer-grained approach: instead of partitioning the model at the layer boundary, it partitions the weight matrices within individual layers across multiple devices. For a transformer attention layer with a large weight matrix W:

- GPU 0 — holds columns 1 to W/2 of the weight matrix
- GPU 1 — holds columns W/2+1 to W of the weight matrix

Both GPUs compute their respective portions of the matrix multiplication in parallel, then communicate to combine results via an All-Reduce operation. This approach exploits the inherent mathematical decomposability of matrix operations (matrix multiplication is column-decomposable) to achieve fine-grained parallelism.

Tensor parallelism is the approach used by Megatron-LM (NVIDIA's framework for training billion-parameter models) and is typically combined with pipeline parallelism in production large-model training systems. It achieves very high GPU utilization but requires high-bandwidth interconnects (NVLink) because it generates frequent, small communication operations between every pair of tensor-parallel GPUs.

3.3 Data Parallelism vs. Model Parallelism — A Direct Comparison

Data Parallelism	Model Parallelism
Full model on every GPU	Model split across GPUs
Dataset split across GPUs	Same data flows through all devices
Lower communication frequency	High communication frequency
Near-linear throughput scaling	Sub-linear scaling (pipeline/tensor overhead)
Simple to implement (DDP wrapper)	Complex implementation
Requires model to fit in single GPU	Enables models larger than any single GPU
Best when: model fits in GPU, data is large	Best when: model exceeds single GPU memory

3.4 Fully Sharded Data Parallelism (FSDP) — The Modern Hybrid

PyTorch's Fully Sharded Data Parallel (FSDP) represents the state-of-the-art approach to scaling training when models are large but not excessively so (roughly 7B–70B parameters).

FSDP shards not just the data but also the model parameters, gradients, and optimizer states across all participating GPUs — making the memory usage per GPU proportional to 1/N of the total where N is the number of GPUs.

The key innovation of FSDP is its lazy parameter assembly strategy: sharded parameters are only reconstructed into full tensors immediately before they are needed for computation, then immediately re-sharded afterward. This means GPU memory is never occupied by more than the parameters being actively computed at any moment, enabling extremely memory-efficient training.

Approach	Parameters per GPU	Gradients per GPU	Optimizer State per GPU	Best For
DDP	Full model (N copies)	Full gradient	Full optimizer state	Model fits in GPU
FSDP	1/N of model	1/N of gradients	1/N of optimizer state	Large models (7B–70B)
Megatron-LM	1/N of model	1/N + pipeline overhead	Distributed	Massive models (100B+)
DeepSpeed ZeRO	1/N of everything	1/N of gradients	Partitioned	Flexible, all scales

Chapter 4

Hybrid Parallelism

Combining strategies to train the world's largest AI models

4.1 Why Hybrid Is Necessary

Neither data parallelism nor model parallelism alone is sufficient for training the largest frontier models. Data parallelism requires the model to fit in a single GPU's VRAM — an impossible condition for models with hundreds of billions of parameters. Model parallelism alone dramatically limits effective throughput, because a single model shard on a single GPU can only process one stream of data at a time.

Hybrid parallelism combines both strategies simultaneously, deploying each where it is most effective. The resulting system achieves memory capacity (from model parallelism) and computational throughput (from data parallelism) simultaneously — enabling the training of models at scales that neither strategy could achieve independently.

> **Definition**
> Hybrid parallelism simultaneously applies multiple parallelism strategies within the same training cluster. The most common combination uses tensor parallelism within nodes (exploiting NVLink's high bandwidth for frequent intra-layer communications), pipeline parallelism across nodes (partitioning the model's depth with lower-frequency inter-node communications), and data parallelism across node groups (replicating the combined tensor+pipeline-parallel model for throughput).

4.2 3D Parallelism — The Standard for Trillion-Parameter Training

The dominant architecture for training the largest models is called 3D Parallelism, pioneered by Microsoft's DeepSpeed team and adopted by NVIDIA's Megatron-LM. It combines all three parallelism dimensions simultaneously:

Parallelism Dimension	Partitions	Communication Pattern	Typical Scale
Tensor Parallelism	Weight matrices within layers	High-frequency, low-volume (NVLink)	2–8 GPUs per node
Pipeline Parallelism	Sequential layer groups	Medium-frequency, medium-volume	4–64 stages across nodes

Parallelism Dimension	Partitions	Communication Pattern	Typical Scale
Data Parallelism	Training data mini-batches	Low-frequency, high-volume (IB)	Hundreds–thousands of nodes

A concrete example from the training of GPT-3-scale models: a cluster of 1,024 GPUs arranged as 8-way tensor parallelism within each 8-GPU node, 16-way pipeline parallelism across 16 nodes, and 8-way data parallelism across 8 groups of 16 nodes. This three-dimensional arrangement coordinates all 1,024 GPUs effectively while managing communication overhead at each level.

4.3 Communication Hierarchy in Hybrid Parallelism

A critical engineering principle in hybrid parallel training is matching each parallelism strategy to the appropriate communication tier in the cluster network hierarchy:

- Tensor parallelism — deployed within a single server node, communicating over NVLink (600 GB/s bandwidth). The high frequency and moderate volume of tensor-parallel communications require NVLink's extraordinary bandwidth — InfiniBand (100 Gb/s) would create a severe bottleneck.
- Pipeline parallelism — deployed across server nodes within a data center rack, communicating over high-speed InfiniBand (200 Gb/s or 400 Gb/s HDR/NDR). Activation tensors between pipeline stages are large but infrequent — matching InfiniBand's characteristics.
- Data parallelism — deployed across node groups within or between data center pods, communicating over cluster InfiniBand fabric. All-Reduce gradient synchronization is infrequent (once per training step) but involves the largest data volumes — benefiting from InfiniBand's reliability for large transfers.

Chapter 5

Challenges in Distributed Training

The engineering problems that make distributed AI hard

5.1 Communication Overhead — The Dominant Cost

In an ideal world, distributing training across N GPUs would provide exactly N× speedup. In practice, the achieved speedup is always less — often significantly less for very large clusters — because of communication overhead. Every synchronization point between GPUs requires time during which GPUs cannot compute, creating idle periods that directly reduce effective GPU utilization.

Communication overhead is proportional to:

- Model size — larger models generate more gradient data per synchronization step
- Number of GPUs — more participants means more communication rounds in collective operations
- Network topology — the physical and logical arrangement of GPUs affects collective operation efficiency
- Network bandwidth — InfiniBand (200–400 Gb/s) dramatically outperforms Ethernet (10–100 Gb/s) for distributed training workloads

Communication Strategy	Latency	Bandwidth Efficiency	Scalability
Synchronous All-Reduce	Highest (all GPUs wait)	Excellent	Limited (straggler problem)
Asynchronous SGD	Lowest (no waiting)	Good	High (no synchronization barrier)
Gradient Compression	Medium	Very High	High (reduced data volume)
Local SGD (periodic sync)	Low	High	High (less frequent communication)

5.2 The Straggler Problem

Synchronous distributed training requires all GPUs to reach the synchronization barrier before the next training step can begin. This creates a vulnerability to stragglers — individual GPUs that are slower than their peers due to hardware variation, thermal throttling, memory fragmentation, or background system processes.

The operational impact is severe: if one GPU out of 1,000 is 10% slower than average, the entire training step takes 10% longer than it should — effectively reducing the compute efficiency of all 999 other GPUs by 10%. At large cluster scales, the probability of encountering at least one straggler per training step approaches certainty.

Mitigation strategies include:

- Backup workers — launch more replicas than required and use the first N to complete each step
- Asynchronous training — allow workers to use slightly stale gradient information rather than waiting for synchronization
- Hardware monitoring — proactively detect and replace underperforming GPUs before they become stragglers
- Fault-tolerant training — checkpoint frequently and restart failed workers without losing the training run

5.3 Load Balancing

Distributed training achieves maximum efficiency when every GPU in the cluster performs exactly the same amount of work in each training step. When work is unevenly distributed — some GPUs processing more data, more parameters, or more complex computations than others — the faster GPUs idle while waiting for the slower ones, wasting expensive GPU compute time.

Load imbalance manifests differently across parallelism strategies:

- Data parallelism — variable-length sequences (e.g., text documents of different lengths) create uneven GPU loads if not carefully sorted and batched
- Pipeline parallelism — unequal layer counts or computation costs across pipeline stages create 'hot stages' that bottleneck the entire pipeline
- Tensor parallelism — irregular weight matrix shapes that don't divide evenly across the tensor-parallel degree create padding overhead

5.4 Fault Tolerance and Checkpoint Management

Training large models takes weeks to months of continuous GPU cluster operation. During this time, hardware failures are not edge cases — they are statistical certainties. A cluster of 1,000 GPUs with individual GPU MTBF (mean time between failures) of 1 year will experience on average ~2.7 GPU failures per day. Without fault tolerance mechanisms, any single hardware failure would abort the entire training run.

Production distributed training systems address this with:

- Checkpoint-based recovery — periodically save complete model state (parameters, optimizer state, training step) to persistent storage; restart from last checkpoint after failure

- Elastic training — dynamically add or remove workers from the training cluster without stopping the run
- Automatic failover — detect failed workers and redistribute their workload to healthy nodes
- Mixed-precision checkpointing — save both FP32 master weights and FP16 training weights for recovery

Engineering Rule of Thumb

For training runs expected to last more than 24 hours, checkpoint at least every 15–30 minutes. For training runs on clusters of 500+ GPUs, checkpoint at least every 10 minutes. Each checkpoint should be verified (written successfully and readable) before the training step is considered complete. Never rely on a single checkpoint location — replicate to at least two storage destinations to protect against storage system failures.

Ray Framework

The Ray Ecosystem

Python-native distributed computing from laptop to thousand-node cluster

6.1 What Is Ray?

Ray is an open-source distributed computing framework developed at UC Berkeley's RISELab (now the Sky Computing Lab) and maintained by Anyscale. It provides a general-purpose Python API for distributed and parallel computation — from simple task parallelism to complex distributed ML training pipelines — while abstracting away the underlying cluster management, network communication, and scheduling complexity.

Ray's defining characteristic is its seamless scalability: code written to run on a developer's laptop runs without modification on a 1,000-node cluster. The programmer writes standard Python, decorates functions with @ray.remote, and Ray handles the rest — scheduling tasks across available nodes, managing distributed object storage, handling failures, and optimizing communication patterns.

Ray's Core Design Philosophy

Ray was built around a single guiding principle: the same Python code should work everywhere, from one machine to many. This 'write once, run anywhere' philosophy distinguishes Ray from traditional distributed computing frameworks (like Apache Spark or Dask) that require programmers to restructure their code around framework-specific abstractions. With Ray, you write Python functions and objects — Ray makes them distributed.

6.2 The Ray Ecosystem Architecture

Ray is not a single tool but an ecosystem of libraries built on a common distributed execution runtime. Each component addresses a specific phase of the AI development and deployment lifecycle:

Ray Component	Primary Purpose	Key Use Cases	Analogous Technology
Ray Core	Distributed task and actor execution	Parallel data processing, custom distributed logic	Python multiprocessing + RPC
Ray Data	Scalable dataset processing	Data loading, preprocessing, feature engineering	Apache Spark DataFrames

Ray Component	Primary Purpose	Key Use Cases	Analogous Technology
Ray Train	Distributed model training	DDP, FSDP, mixed-precision, multi-GPU training	PyTorch DDP + Horovod
Ray Tune	Hyperparameter optimization	Parallel HPO, early stopping, search algorithms	Optuna + parallel execution
Ray Serve	Model serving and deployment	Real-time inference, autoscaling, multi-model	TorchServe + FastAPI
Ray RLlib	Reinforcement learning	RLHF, policy training, multi-agent RL	Stable-Baselines3 + distributed

6.3 Ray Core — The Execution Foundation

6.3.1 Tasks — Distributed Functions

Ray's fundamental building block for stateless distributed computation is the remote task. Any Python function can be converted into a Ray task by decorating it with @ray.remote. When called with .remote(), the function is scheduled for execution on any available worker in the Ray cluster — potentially on a completely different machine than the caller.

The interaction model is asynchronous by default: task_ref = expensive_function.remote() returns immediately with an ObjectRef (a future/promise) while the task executes in the background. ray.get(task_ref) retrieves the result, blocking only at this point. This enables sophisticated parallel execution patterns where many tasks are submitted and their results collected in bulk.

Key characteristics of Ray tasks:

- Stateless — each task invocation is independent; tasks don't share state between calls
- Fault-tolerant — if the worker executing a task fails, Ray automatically retries the task on another worker
- Resource-aware — tasks can declare GPU and CPU requirements; Ray's scheduler only assigns tasks to workers with sufficient resources
- Composable — task futures can be passed as arguments to other tasks, enabling dependency-aware distributed pipelines

6.3.2 Actors — Stateful Distributed Objects

While tasks handle stateless computation, many real-world AI applications require shared mutable state — a parameter server holding model weights, a replay buffer for reinforcement learning, a shared feature cache, or a coordination service for distributed training. Ray Actors address this need.

An Actor is a Python class decorated with @ray.remote that is instantiated as a long-lived process on a cluster node. All method calls on the Actor are executed sequentially within that single process, providing a simple, consistent concurrency model. Unlike tasks, Actors maintain state between method calls — their instance variables persist across invocations.

Actor characteristics:

- Stateful — instance variables persist between method calls; ideal for shared mutable data structures
- Location-pinned — an Actor runs on one specific node for its entire lifetime (by default)
- Handle-based access — Actor handles can be passed to any task or other Actor in the cluster
- Schedulable — Ray can place Actors on specific nodes or let the scheduler choose optimally

6.3.3 Object Store — Distributed Shared Memory

Ray includes a distributed shared memory system called the Object Store, implemented using Apache Arrow's Plasma in-memory format. The Object Store enables zero-copy data sharing between Ray tasks and Actors on the same node, and efficient data transfer between nodes.

The two primary APIs for interacting with the Object Store directly are:

- ray.put(object) — serializes a Python object into the Object Store and returns an ObjectRef. This is useful for large objects (datasets, model weights) that will be referenced by many tasks — storing once in the Object Store is far more efficient than serializing and transmitting the object separately to each task.
- ray.get(ref) — deserializes the object referenced by an ObjectRef back into a Python object. If the object is stored on a remote node, Ray transfers it to the local node first.

6.4 Ray Tune — Hyperparameter Optimization at Scale

Hyperparameter optimization (HPO) is one of the most computationally demanding and parallelizable components of the ML workflow. For any given model architecture, dozens to hundreds of hyperparameters control training dynamics: learning rate, batch size, weight decay, dropout rates, architecture-specific parameters like attention head count, layer widths, and activation functions.

Finding optimal hyperparameters through grid search is computationally intractable for modern deep learning models — the search space is too large. Ray Tune enables intelligent, parallel HPO by distributing many concurrent trials across available cluster resources, applying early stopping to unpromising configurations, and using Bayesian optimization to focus the search on promising regions of the hyperparameter space.

Search Algorithm	Strategy	Best For
Random Search	Sample configurations uniformly at random	Quick baselines, highly parallel, surprisingly effective
Bayesian Optimization	Model the performance landscape probabilistically	Sample-efficient when trials are expensive
HyperBand (ASHA)	Early stopping of underperforming trials	Very large search spaces, cheap to evaluate
Population-Based Training	Evolve hyperparameters during training	Online adaptation, RLHF, dynamic schedules

Ray Tune integrates natively with major ML frameworks (PyTorch, TensorFlow, XGBoost, scikit-learn) and search libraries (Optuna, HyperOpt, Ax), making it a flexible hyperparameter optimization engine regardless of the underlying model technology. Its native Ray integration means HPO trials automatically scale across any available cluster resources without additional configuration.

6.5 Ray Train — Distributed Model Training

Ray Train is Ray's library for distributed deep learning training, supporting all major parallelism strategies (data parallelism, model parallelism, and their combinations) through a consistent, framework-agnostic API. Its core value proposition is enabling distributed training with minimal code changes — a model written for single-GPU training can typically be converted to multi-GPU distributed training by changing fewer than 10 lines of code.

Ray Train provides first-class integrations for:

- PyTorch DDP — wraps models in DistributedDataParallel for data-parallel training
- PyTorch FSDP — coordinates Fully Sharded Data Parallel training across large GPU clusters
- DeepSpeed — integrates Microsoft's ZeRO optimizer stages for ultra-large model training
- TensorFlow — distributed training via tf.distribute strategies
- Hugging Face Accelerate — integration with the HuggingFace distributed training ecosystem

A key operational advantage of Ray Train is its built-in fault tolerance: failed training workers are automatically detected and restarted, resuming from the most recent checkpoint without human intervention. This is particularly valuable for long-running training jobs on cloud spot/preemptible instances, where worker interruptions are expected.

6.6 Ray Serve — Production Model Serving

Ray Serve is a scalable model serving library built on Ray's distributed actor system. Unlike traditional model serving frameworks that require models to be packaged in specific formats, Ray Serve serves any Python callable — a HuggingFace pipeline, a scikit-learn model, a LangChain chain, or a custom neural network inference function.

Ray Serve production features:

- Horizontal autoscaling — automatically adds or removes serving replicas based on request queue depth and latency
- Request batching — intelligently batches concurrent requests for GPU efficiency
- Traffic routing — supports A/B testing, canary deployments, and gradual rollouts between model versions
- Multi-model composition — chains multiple models into inference pipelines with efficient inter-model communication
- HTTP and gRPC support — serves models over standard web protocols compatible with any client

6.7 Companies Using Ray at Scale

Organization	Ray Use Case	Scale
OpenAI	Scalable reinforcement learning (RLHF, PPO training for GPT models)	Thousands of GPUs
Netflix	Large-scale recommendation system training and feature engineering	Petabytes of data
Uber	Deep learning infrastructure for maps, demand forecasting, routing AI	Enterprise scale
Shopify	Demand forecasting, fraud detection, recommendation systems	Millions of merchants
Instacart	Real-time recommendations, search ranking, delivery optimization	Millions of daily orders
Cohere	Large language model training and fine-tuning infrastructure	Foundation model scale

Chapter 7

Anyscale

Enterprise-grade Ray — the production platform for distributed AI workloads

7.1 What Is Anyscale?

Anyscale is the commercial company founded by the Ray creators (Ion Stoica, Robert Nishihara, Philipp Moritz, and others from UC Berkeley's RISELab). Anyscale provides a fully managed cloud platform built on Ray that handles all the infrastructure complexity of running distributed AI workloads at enterprise scale — cluster provisioning, auto-scaling, monitoring, cost optimization, security, and compliance.

The relationship between Ray and Anyscale parallels other open-source/commercial pairs in the data ecosystem: Ray is the open-source framework (like Kubernetes or Apache Spark); Anyscale is the managed platform built on top of it (like GKE or Databricks). Organizations can use Ray directly and manage their own clusters, or use Anyscale to eliminate that operational burden.

7.2 Key Anyscale Capabilities

Capability	Description	Business Value
Managed Ray Clusters	Fully automated cluster lifecycle — provision, scale, repair	No infrastructure engineering team needed
Job Scheduling	Submit Ray jobs with dependencies, priorities, and resource quotas	Reproducible, auditable training runs
Workspaces	Cloud-hosted development environments with cluster access	Develop and test distributed code easily
Anyscale Services	Managed Ray Serve deployments with autoscaling and monitoring	Production model serving without ops burden
Cost Management	Spot instance integration, utilization monitoring, cost dashboards	Minimize cloud spend on GPU-intensive workloads
Multi-Cloud	Deploy across AWS, GCP, and Azure from a single interface	Avoid vendor lock-in, optimize by region pricing

7.3 Ray Open Source vs. Anyscale — Decision Framework

☑ Choose Anyscale When	☑ Choose Self-Managed Ray When
No infrastructure management overhead	Full infrastructure control
Enterprise SLAs and support	No per-seat or platform fees
Built-in cost optimization tooling	No vendor lock-in
Compliance and security controls	Custom networking and security
Rapid deployment path	Open-source community support
Vendor-managed lifecycle	Requires dedicated ML infra team

Section 2

Practical Implementation & Coding Walkthrough

Complete cell-by-cell explanation — building distributed training and parallel computation with Ray

Chapter 8

Building Distributed AI Workloads with Ray

From installation to distributed training — every line explained

This section walks through a complete practical implementation of distributed computing with Ray — covering installation, core task and actor patterns, Ray Data for scalable data processing, Ray Tune for parallel hyperparameter optimization, and Ray Train for distributed model training. Every code cell is explained in depth, connecting the mechanics to the conceptual foundations from Section 1.

Cell 1 — Installing Ray and Related Libraries

What This Cell Does

Installs the Ray ecosystem along with the libraries needed for distributed ML training and hyperparameter optimization — the complete foundation for building distributed AI systems in Python.

```
pip install ray[data,train,tune,serve] torch torchvision scikit-learn
```

Package-by-Package Explanation

ray[data,train,tune,serve] — The Ray Ecosystem

The base ray package installs Ray Core — the distributed task and actor execution engine. The bracketed extras install optional components:

- ray[data] — Ray Data: scalable, lazy dataset processing with Arrow backend; parallel data preprocessing and feature engineering across clusters
- ray[train] — Ray Train: distributed deep learning training integrations for PyTorch DDP, FSDP, DeepSpeed, and TensorFlow
- ray[tune] — Ray Tune: hyperparameter optimization with support for Bayesian optimization, HyperBand, and population-based training

- ray[serve] — Ray Serve: production model serving with autoscaling, batching, and multi-model composition

Installing all extras together ensures compatibility between components — they share the same Ray Core version and communicate through the same distributed object store and scheduler.

torch and torchvision — PyTorch Deep Learning Stack

PyTorch is the dominant deep learning framework for research and production. torchvision provides datasets (CIFAR-10, ImageNet), pretrained vision models (ResNet, VGG, ViT), and image transformation utilities. Together, they provide the complete stack for neural network definition, training, and evaluation.

Ray Train's PyTorch integration operates through a thin wrapper around PyTorch's native DistributedDataParallel — meaning any standard PyTorch training loop can be scaled to multiple GPUs or nodes with minimal modification. The PyTorch code remains idiomatic; Ray handles the distributed coordination.

scikit-learn — Classical ML for Baseline Experiments

scikit-learn provides a comprehensive library of classical machine learning algorithms (random forests, SVMs, gradient boosting, dimensionality reduction) along with preprocessing utilities and evaluation metrics. It is included here because Ray Tune has excellent scikit-learn integration — enabling parallel hyperparameter search over scikit-learn estimators using the same API used for deep learning models.

Cell 2 — Initializing the Ray Runtime

What This Cell Does

Initializes the Ray cluster runtime — either connecting to an existing remote cluster or starting a local cluster using all available CPU and GPU resources on the current machine.

```
import ray

ray.init()
```

Line-by-Line Explanation

import ray — The Root Import

Imports the Ray Python package, making all Ray APIs available. This import triggers no distributed computation — it simply loads the Python module. The Ray runtime is not started until ray.init() is called.

ray.init() — Starting or Connecting to the Ray Runtime

ray.init() is the most important Ray API call. It performs several critical operations:

1. Starts the Ray head node process (if no cluster exists) — the cluster's scheduler and object store coordinator
2. Launches worker processes — one per available CPU core by default
3. Initializes the distributed object store (Plasma) — shared memory for zero-copy data sharing
4. Starts the GCS (Global Control Service) — manages cluster metadata and object locations
5. Opens the Ray Dashboard — a web UI for monitoring task execution, GPU utilization, and cluster health

ray.init() is context-aware: if called on a developer's laptop, it creates a local multi-process cluster using available CPUs. If called within an Anyscale workspace or with a cluster address parameter, it connects to the remote cluster. This is the mechanism that enables the 'write once, run anywhere' promise — the same ray.init() call works on a MacBook and on a 1,000-node cloud cluster.

Key configuration options:

- ray.init() — auto-detect local resources, start local cluster
- ray.init(num_cpus=8, num_gpus=2) — override resource detection for testing
- ray.init(address="auto") — connect to existing Ray cluster via standard discovery
- ray.init(address="ray://cluster-head:10001") — connect to specific remote cluster

> ⚠ **ray.init() Best Practices**
> Call ray.init() only once per Python process. Calling it multiple times without an intervening ray.shutdown() raises an error. In Jupyter notebooks, use ray.is_initialized() to check whether a cluster is already running before calling ray.init(). In production scripts and Ray Job submissions, ray.init() is typically called at the top of the main() function.

Cell 3 — Creating and Executing Ray Remote Tasks

> **What This Cell Does**
> Defines a Ray remote function that executes a computation in parallel on the distributed Ray cluster, submits it for asynchronous execution, and retrieves the result using ray.get().

```
import ray

@ray.remote
def compute_square(x):
```

```
    return x * x

# Submit tasks asynchronously
futures = [compute_square.remote(i) for i in range(10)]

# Retrieve results
results = ray.get(futures)
print('Squares:', results)
```

Line-by-Line Explanation

@ray.remote — The Distribution Decorator

The @ray.remote decorator transforms a regular Python function into a Ray remote function. This single decorator causes a fundamental behavioral change: instead of executing synchronously in the current process when called, the function is serialized (using cloudpickle), sent to a Ray worker process (potentially on a different machine), executed there, and the result is stored in the distributed object store.

The decorator does not change the function's Python signature or behavior — it only changes where and how the function executes. This minimal-intrusion design is a core Ray philosophy: existing Python code can be distributed by adding a single decorator, without rewriting business logic.

def compute_square(x): return x * x — The Task Function

The task function itself is ordinary Python. It receives arguments (serialized by Ray when submitted), executes computation, and returns a value (serialized by Ray and stored in the object store). There are no Ray-specific APIs inside the function — it could run identically as a regular Python function in a non-distributed context.

compute_square.remote(i) — Asynchronous Task Submission

Calling .remote() instead of calling the function directly submits the task to the Ray scheduler. The call returns immediately with an ObjectRef — a handle to the future result. The actual computation happens asynchronously on a Ray worker. This is the fundamental Ray pattern: submit first, retrieve later.

The list comprehension [compute_square.remote(i) for i in range(10)] submits 10 tasks to the Ray scheduler simultaneously. Ray schedules them across available workers in parallel — if 10 workers are available, all 10 computations run concurrently. This is the basic data parallelism pattern at the task level.

futures = [...] — Collecting ObjectRefs

futures is a list of 10 ObjectRefs — handles to pending or completed computation results stored in Ray's distributed object store. ObjectRefs are lightweight (a few hundred bytes each) and can

be stored, passed between processes, and used as arguments to other remote tasks — enabling complex dependency graphs of distributed computation.

ray.get(futures) — Materializing Results

ray.get() retrieves the actual Python objects referenced by a list of ObjectRefs. This call blocks until all referenced computations are complete and all results are available in the object store. For results on remote nodes, Ray transfers them to the local node before deserializing.

The call pattern matters for performance. ray.get([ref1, ref2, ref3]) is more efficient than [ray.get(ref1), ray.get(ref2), ray.get(ref3)] because the batch form allows Ray to optimize data transfers and overlaps deserialization work. Never call ray.get() inside a loop over individual ObjectRefs — always collect all refs first and call ray.get() once with the list.

Cell 4 — Building Stateful Distributed Systems with Ray Actors

What This Cell Does

Defines a Ray Actor class that maintains persistent state across method calls, instantiates it as a distributed process on the Ray cluster, and interacts with it through remote method calls.

```
import ray

@ray.remote
class ParameterServer:
    def __init__(self):
        self.parameters = {}

    def push(self, key, value):
        self.parameters[key] = value

    def pull(self, key):
        return self.parameters.get(key)

# Create the Actor on the Ray cluster
ps = ParameterServer.remote()

# Push a gradient update
ps.push.remote("layer_1_weights", [0.01, -0.02, 0.03])

# Pull the current value
result = ray.get(ps.pull.remote("layer_1_weights"))
print('Retrieved:', result)
```

Line-by-Line Explanation

@ray.remote on a Class — Creating an Actor

When @ray.remote decorates a class (rather than a function), it defines an Actor class. Unlike the function case (which creates stateless tasks), the class case creates a stateful distributed object. The class definition itself is unchanged — any Python class can become a Ray Actor by adding this decorator.

class ParameterServer — The Parameter Server Pattern

The Parameter Server is a foundational distributed systems pattern in ML training. It provides a centralized store for model parameters that is accessible from all distributed training workers. Workers push gradient updates to the parameter server after computing them locally, and pull the latest parameter values before each forward pass.

This pattern is the basis for asynchronous distributed training: workers don't wait for each other to synchronize gradients; they independently pull and push parameters at their own pace. The parameter server absorbs the coordination complexity, allowing workers to maximize GPU utilization.

self.parameters = {} — Persistent State

This dictionary persists for the entire lifetime of the Actor process. Unlike a Ray task (where local variables are destroyed after the function returns), Actor instance variables survive between method calls. This is the fundamental property that distinguishes Actors from tasks — they maintain state.

In production parameter server implementations, this dictionary would be replaced by a more sophisticated data structure: a distributed tensor store with versioning, consistency guarantees, and efficient bulk transfer capabilities.

ps = ParameterServer.remote() — Instantiating the Actor

ParameterServer.remote() creates a new Actor instance on the Ray cluster — allocating a dedicated process on an available worker node, executing __init__(), and returning an Actor handle. This handle (ps) can be stored, passed to other tasks and actors, and used from any node in the cluster — not just the node where the Actor is running.

ps.push.remote(...) and ps.pull.remote(...) — Asynchronous Method Calls

Method calls on Actors use the same .remote() syntax as task submissions. ps.push.remote('layer_1_weights', [...]) submits the push method call to the Actor's task queue and returns an ObjectRef. The Actor's method calls execute sequentially in submission order — Ray guarantees that push completes before a subsequent pull begins for the same Actor.

This sequential execution guarantee is important: it provides a simple, consistent memory model for Actor state without requiring locks or other concurrency primitives. All method calls to a single Actor are automatically serialized by the Ray scheduler.

Cell 5 — Scalable Data Processing with Ray Data

What This Cell Does

Demonstrates Ray Data's API for parallel, scalable data processing — loading datasets, applying transformations in parallel across the cluster, and computing aggregations without loading all data into a single process.

```
import ray

# Create a Ray Dataset from an in-memory list
dataset = ray.data.from_items([
    {'text': 'positive review', 'label': 1},
    {'text': 'negative review', 'label': 0},
    {'text': 'great product',   'label': 1},
    {'text': 'terrible item',   'label': 0},
])

# Apply a parallel transformation
def add_length(record):
    record['text_length'] = len(record['text'])
    return record

transformed = dataset.map(add_length)

# Show results
transformed.show()
```

Line-by-Line Explanation

ray.data.from_items([...]) — Creating a Dataset

ray.data.from_items() creates a Ray Dataset from a Python list. In production pipelines, datasets are created from more scalable sources: ray.data.read_csv() for CSV files on S3 or GCS, ray.data.read_parquet() for columnar Parquet files, ray.data.read_json() for JSON Lines files, or ray.data.from_huggingface() for HuggingFace datasets.

A Ray Dataset is fundamentally different from a pandas DataFrame or a Python list. It is a distributed, lazy data structure partitioned across all nodes in the Ray cluster. Data operations on a Ray Dataset execute in parallel across all partitions simultaneously — processing petabytes of data without any single node needing to hold the full dataset in memory.

dataset.map(add_length) — Parallel Transformation

map() applies the add_length function to every record in the dataset. Critically, this transformation executes in parallel across all partitions of the dataset on all available cluster nodes simultaneously. For a dataset with 10 million records partitioned across 100 nodes, each

node processes ~100,000 records in parallel — achieving 100× the throughput of single-node processing.

Ray Data's map() is lazy by default — it doesn't execute until results are explicitly requested (by calling .show(), .take(), .to_pandas(), or by feeding the dataset into a training loop). This lazy evaluation enables Ray Data to optimize entire data pipelines before executing them — fusing compatible operations, choosing optimal partition counts, and co-locating transformations with data to minimize network transfers.

transformed.show() — Materializing Results

show() triggers execution of the full transformation pipeline and prints the first few records to the console. Other materialization methods include: take(n) to retrieve the first n records as a list, to_pandas() to convert to a pandas DataFrame (on single machines), to_torch() to create a PyTorch DataLoader iterator, and iter_batches() to stream data in mini-batches through a training loop.

Cell 6 — Parallel Hyperparameter Optimization with Ray Tune

What This Cell Does

Defines a training function that Ray Tune will execute as many parallel trials, each with different hyperparameter configurations sampled from a search space — automatically reporting metrics and finding the best configuration.

```
from ray import tune
from ray.tune.schedulers import ASHAScheduler

def train_model(config):
    # Simulated training loop
    learning_rate = config['lr']
    batch_size    = config['batch_size']

    # In practice: real model training here
    accuracy = 0.85 + (learning_rate * 10) - (batch_size * 0.001)

    # Report metric to Ray Tune
    tune.report(accuracy=accuracy)

# Define the hyperparameter search space
search_space = {
    'lr':         tune.loguniform(1e-4, 1e-1),
    'batch_size': tune.choice([16, 32, 64, 128]),
}

# Launch parallel hyperparameter search
tuner = tune.Tuner(
    train_model,
    param_space=search_space,
```

```
    tune_config=tune.TuneConfig(
        num_samples=20,
        scheduler=ASHAScheduler(metric='accuracy', mode='max'),
    ),
)
results = tuner.fit()
print('Best config:', results.get_best_result().config)
```

Block-by-Block Explanation

from ray.tune.schedulers import ASHAScheduler

ASHAScheduler implements the Asynchronous Successive Halving Algorithm — an early stopping strategy that aggressively terminates poorly-performing trials, reallocating their computational resources to more promising ones. ASHA is the recommended scheduler for most HPO use cases because it achieves near-optimal results with dramatically fewer total training steps than random search without early stopping.

The algorithm works by running all trials initially, then periodically evaluating performance and terminating the bottom fraction of trials. Survivors proceed to the next 'rung,' receiving more compute. This creates an efficient funnel that concentrates resources on the most promising hyperparameter configurations.

def train_model(config) — The Trainable Function

The trainable function is the core of any Ray Tune experiment. It receives a config dictionary (populated by Ray Tune with specific hyperparameter values for this trial), executes the training loop, and reports metrics using tune.report(). The function's structure is intentionally simple — it is an ordinary Python function that Ray Tune executes as a remote task.

In a real training scenario, this function would:

6. Initialize the model with architecture parameters from config
7. Create the optimizer using config['lr'] and config['weight_decay']
8. Load the dataset and create DataLoaders with config['batch_size']
9. Execute the training loop for config['num_epochs'] epochs
10. Call tune.report(accuracy=val_accuracy, loss=val_loss) after each epoch

tune.loguniform(1e-4, 1e-1) — Log-Scale Sampling

loguniform samples learning rate values on a logarithmic scale between 1e-4 (0.0001) and 1e-1 (0.1). Log-scale sampling is critically important for learning rate: a uniform sample would heavily over-represent large values (most samples would be between 0.05 and 0.1) and under-represent small values. Log-scale ensures equal representation across orders of magnitude, which matches the true structure of the learning rate search problem.

tune.choice([16, 32, 64, 128]) — Categorical Sampling

choice samples uniformly from a discrete set of options. Batch size is a natural discrete hyperparameter — you use 32 or 64, not 47.3. Other discrete distributions in Ray Tune include tune.randint(low, high) for integers and tune.grid_search([...]) for exhaustive grid search over a parameter.

num_samples=20 — Total Trial Count

num_samples specifies how many distinct hyperparameter configurations to evaluate. With num_samples=20 and a 4-worker cluster, Ray Tune runs 4 trials simultaneously, starting new trials as existing ones complete. All 20 trials execute in less than 5× the time of a single trial (rather than 20× with sequential search). With ASHA early stopping, many trials terminate early — the effective compute budget is much less than 20 full training runs.

results.get_best_result().config — Retrieving the Winner

After all 20 trials complete, get_best_result() retrieves the trial that achieved the highest accuracy (because mode='max' in ASHAScheduler). The .config attribute returns the exact hyperparameter dictionary that produced the best result — ready to be used for a final full training run with the optimal configuration.

Cell 7 — Distributed Model Training with Ray Train

What This Cell Does

Defines a complete distributed training loop using Ray Train's TorchTrainer — automatically distributing model training across multiple GPUs or machines with data parallelism, gradient synchronization, and fault-tolerant checkpointing.

```
import ray.train.torch
from ray.train.torch import TorchTrainer
from ray.train import ScalingConfig
import torch
import torch.nn as nn

# Define the training loop (runs on each worker)
def train_loop_per_worker(config):
    # Prepare model for distributed training
    model = nn.Linear(10, 1)          # Simple linear model
    model = ray.train.torch.prepare_model(model)

    optimizer = torch.optim.SGD(
        model.parameters(), lr=config['lr']
    )
    loss_fn = nn.MSELoss()

    # Training loop
    for epoch in range(config['num_epochs']):
        X = torch.randn(100, 10)      # Synthetic features
        y = torch.randn(100, 1)       # Synthetic targets
```

```
        # Prepare data for distributed training
        X, y = ray.train.torch.prepare_data_loader(
            [(X, y)]
        )

        optimizer.zero_grad()
        pred  = model(X[0])
        loss  = loss_fn(pred, y[0])
        loss.backward()
        optimizer.step()

    # Report final loss
    ray.train.report({'loss': loss.item()})

# Configure distributed training
trainer = TorchTrainer(
    train_loop_per_worker=train_loop_per_worker,
    train_loop_config={'lr': 0.01, 'num_epochs': 5},
    scaling_config=ScalingConfig(
        num_workers=4,
        use_gpu=True,
    ),
)

result = trainer.fit()
print('Training complete. Metrics:', result.metrics)
```

Block-by-Block Explanation

def train_loop_per_worker(config) — The Per-Worker Training Function

This function defines the training logic that executes on every worker in the distributed training setup. If ScalingConfig specifies num_workers=4, this function runs simultaneously on 4 separate processes — potentially on 4 different machines. Each worker trains the same model architecture on different data shards, with gradients automatically synchronized between them.

The function's design follows a critical pattern: all Ray Train APIs (prepare_model, prepare_data_loader, report) abstract the distributed mechanics, while the core PyTorch code (model definition, optimizer, loss computation, backward pass) is completely standard. An engineer familiar with single-GPU PyTorch training can read and understand this function without any distributed systems knowledge.

ray.train.torch.prepare_model(model) — The Distribution Wrapper

prepare_model() is where the distributed magic happens. It wraps the PyTorch model in DistributedDataParallel (DDP) — the standard PyTorch abstraction for data-parallel training. DDP hooks into the model's backward pass, automatically executing an All-Reduce operation on gradients after each backward pass so all workers apply identical gradient updates.

From the perspective of the code after this line, model behaves exactly like a regular PyTorch model — the DDP wrapper is transparent. This transparency is by design: it allows engineers to debug and test training code on a single GPU (without DDP) and then scale to many GPUs by simply adding prepare_model() without changing any other code.

ScalingConfig(num_workers=4, use_gpu=True) — Declaring Resources

ScalingConfig is the resource declaration for the training job. num_workers=4 requests 4 training worker processes. use_gpu=True requests one GPU per worker — so this configuration requests 4 GPUs total. If the Ray cluster has GPUs available across multiple machines, Ray automatically schedules workers on the appropriate nodes.

The declarative resource specification is a fundamental advantage of Ray Train over manual distributed training setup: you declare what you need, Ray figures out where to run it. Changing from 4-GPU to 32-GPU training requires only changing num_workers=4 to num_workers=32 — no changes to the training code, no changes to network configuration, no manual SSH setup.

trainer.fit() — Launching the Distributed Training Job

fit() orchestrates the entire distributed training execution: it provisions the requested workers on the Ray cluster, serializes and distributes the training function to each worker, coordinates the DDP distributed process group initialization, monitors worker health throughout training, handles checkpoint saving, and collects final metrics from all workers.

The return value is a Result object containing the final reported metrics from all workers, the path to the saved checkpoint (if checkpointing was configured), and metadata about the training run (wall-clock time, resource utilization).

Ray Train's Fault Tolerance

If any worker fails during training (hardware failure, preemptible instance interruption, out-of-memory error), Ray Train automatically detects the failure and, depending on configuration, either restarts the failed worker and resumes from the most recent checkpoint, or marks the run as failed and surfaces a clear error. This built-in fault tolerance is one of Ray Train's most important production features — it enables multi-day training runs on spot instances without requiring manual monitoring or intervention.

Cell 8 — Serving Distributed Models with Ray Serve

What This Cell Does

Deploys a trained model as a production-ready inference service using Ray Serve — with autoscaling, request batching, and HTTP endpoint serving backed by the Ray cluster's distributed actor system.

```
from ray import serve
from transformers import pipeline
import requests

@serve.deployment(
    num_replicas=3,
    ray_actor_options={'num_cpus': 2, 'num_gpus': 1},
)
class SentimentClassifier:
    def __init__(self):
        self.model = pipeline('sentiment-analysis')

    async def __call__(self, request):
        data = await request.json()
        result = self.model(data['text'])
        return {'prediction': result}

# Deploy the service to Ray Serve
serve.start()
SentimentClassifier.deploy()

# Test the endpoint
response = requests.post(
    "http://localhost:8000/SentimentClassifier",
    json={"text": "Ray is an amazing distributed computing framework!"}
)
print(response.json())
```

Line-by-Line Explanation

@serve.deployment(num_replicas=3, ray_actor_options=...) — Deployment Configuration

The @serve.deployment decorator transforms a Python class into a Ray Serve deployment — a scalable, distributed service backed by the specified number of Ray Actor replicas. Each replica is an independent Actor process capable of handling inference requests in parallel.

num_replicas=3 provisions 3 independent instances of the SentimentClassifier. Ray Serve load balances incoming HTTP requests across all 3 replicas using round-robin scheduling by default. This provides 3× the throughput of a single instance and tolerates the failure of up to 2 replicas before the service becomes unavailable.

ray_actor_options={'num_cpus': 2, 'num_gpus': 1} declares the resource requirements for each replica. Each of the 3 replicas requires 1 GPU and 2 CPU cores — so this deployment requires 3 GPUs and 6 CPU cores total. Ray Serve only schedules replicas on nodes with sufficient available resources, preventing GPU memory contention between replicas.

class SentimentClassifier — The Deployment Class

The deployment class follows the standard Python class pattern with two key methods: __init__ (executed once when the replica starts, used for expensive initialization like model loading) and __call__ (executed for every incoming inference request).

The model loading in __init__ follows the same principle as the FastAPI example in Chapter 2: load once at startup, not on every request. Each of the 3 Ray Serve replicas loads its own copy of the model into memory (one per GPU), enabling fully parallel inference across all 3 replicas.

async def __call__(self, request) — The Async Request Handler

Making __call__ async enables each replica to handle multiple concurrent requests without blocking. While one request is awaiting GPU inference completion, the async event loop can accept and begin preprocessing another request. This is the same non-blocking concurrency model used by FastAPI — applied here within the Ray Serve actor framework.

serve.start() and SentimentClassifier.deploy() — Launching the Service

serve.start() initializes the Ray Serve controller on the Ray cluster — the central process responsible for managing deployments, routing requests, and monitoring replica health. SentimentClassifier.deploy() provisions the 3 replicas on the cluster and registers the HTTP route /SentimentClassifier.

After deploy() returns, the service is live and accessible at http://localhost:8000/SentimentClassifier (or the cluster's external IP in production). Requests can be sent immediately — Ray Serve handles connection management, request queuing, and response serialization automatically.

Cell 9 — Pure PyTorch Distributed Data Parallel (DDP)

What This Cell Does

Shows the foundational PyTorch Distributed Data Parallel (DDP) pattern — the low-level API underlying Ray Train's distributed training, demonstrating how gradient synchronization actually works at the framework level.

```
import torch
import torch.distributed as dist
import torch.nn as nn
from torch.nn.parallel import DistributedDataParallel as DDP

def setup_distributed(rank, world_size):
    """Initialize the distributed process group."""
    dist.init_process_group(
        backend='nccl',      # NVIDIA's GPU collective operations
        rank=rank,           # This process's global rank (0-indexed)
        world_size=world_size # Total number of processes
    )

def train_ddp(rank, world_size):
    setup_distributed(rank, world_size)
```

```
    # Pin this process to a specific GPU
    device = torch.device(f'cuda:{rank}')

    # Create model and wrap with DDP
    model = nn.Linear(10, 1).to(device)
    ddp_model = DDP(model, device_ids=[rank])

    # Training step
    optimizer = torch.optim.SGD(ddp_model.parameters(), lr=0.01)
    X = torch.randn(32, 10).to(device)
    y = torch.randn(32, 1).to(device)

    optimizer.zero_grad()
    loss = nn.MSELoss()(ddp_model(X), y)
    loss.backward()    # DDP automatically synchronizes gradients here
    optimizer.step()

    print(f'Rank {rank}: loss = {loss.item():.4f}')
    dist.destroy_process_group()
```

Block-by-Block Explanation

import torch.distributed as dist — The Distributed Communication Library

torch.distributed provides PyTorch's distributed communication primitives: All-Reduce, All-Gather, Broadcast, and other collective operations. It is the foundation layer on which DDP, FSDP, and other high-level distributed training abstractions are built. Understanding torch.distributed is essential for debugging distributed training issues and implementing custom communication patterns.

dist.init_process_group(backend='nccl', rank=rank, world_size=world_size)

This call establishes the distributed process group — a communication context shared by all participating processes. The critical parameters:

- backend='nccl' — specifies NVIDIA Collective Communications Library as the communication backend. NCCL is optimized for GPU-to-GPU communication, implementing All-Reduce and other collective operations at near-hardware-bandwidth limits. For CPU-only training, 'gloo' is the appropriate backend.
- rank — the unique integer identifier for this specific process within the distributed group. Rank 0 is conventionally the 'master' process. In 4-GPU training, ranks are 0, 1, 2, and 3.
- world_size — the total number of processes participating in distributed training. With 4 GPUs, world_size=4. This parameter determines how gradients are averaged: each gradient is summed across all processes and divided by world_size.

torch.device(f'cuda:{rank}') — GPU Assignment

Each process is pinned to a specific GPU by using its rank as the CUDA device index. Process 0 uses cuda:0, process 1 uses cuda:1, etc. This ensures no two processes share a GPU,

preventing VRAM contention. Moving the model to the assigned device with .to(device) ensures all model parameters and computations occur on the correct GPU.

DDP(model, device_ids=[rank]) — The Distribution Wrapper

DDP wraps the model and registers hooks on the model's parameters. These hooks are the mechanism of gradient synchronization: immediately after each backward pass, DDP triggers an All-Reduce communication operation that averages the gradients computed by each process. By the time loss.backward() returns, all processes have identical, globally averaged gradients — ready to apply the same optimizer step.

The All-Reduce operation is the performance-critical communication in DDP training. For a model with N parameters at FP32, the All-Reduce transmits 4N bytes of gradient data per training step. With NCCL over NVLink (600 GB/s bandwidth), this is extremely fast for within-node training. Over InfiniBand between nodes, it can become a significant bottleneck for very large models.

loss.backward() — Where Synchronization Happens

This is the single most important line in the DDP training loop. When backward() is called on a DDP-wrapped model, the DDP hooks fire: as soon as all gradients for a parameter bucket (DDP groups parameters into buckets for communication efficiency) are computed, the All-Reduce for that bucket begins immediately — overlapping gradient computation with gradient communication. By the time backward() returns, gradient synchronization is complete across all processes.

This overlap of computation and communication is DDP's most important performance optimization. Without it, every backward pass would be followed by a long communication pause. With it, gradient communication happens in the background during the computation of subsequent parameter gradients — dramatically reducing the effective communication overhead.

> ⚠ **DDP vs. Ray Train Recommendation**
>
> Direct PyTorch DDP as shown in Cell 9 requires manual process group initialization, worker launch scripts (torchrun), and distributed sampler setup. It provides maximum control but substantial boilerplate. For production training jobs, Ray Train (Cell 7) is strongly recommended — it handles all of this automatically, adds fault tolerance, integrates with hyperparameter optimization, and provides a cleaner API. Use raw DDP only when you need custom communication patterns not supported by higher-level frameworks.

Cell 10 — Advanced Hyperparameter Optimization with Population-Based Training

What This Cell Does

Demonstrates Population-Based Training (PBT) — Ray Tune's most sophisticated scheduler, which evolves hyperparameters dynamically during training by exploiting the best-performing trials and mutating their configurations.

```
from ray import tune
from ray.tune.schedulers import PopulationBasedTraining

pbt = PopulationBasedTraining(
    time_attr='training_iteration',
    perturbation_interval=5,
    hyperparam_mutations={
        'lr':           lambda: tune.loguniform(1e-4, 1e-1).sample(),
        'batch_size':   [16, 32, 64],
    },
)

tuner = tune.Tuner(
    trainable=train_model,
    param_space={
        'lr':         tune.loguniform(1e-4, 1e-1),
        'batch_size': tune.choice([16, 32, 64]),
    },
    tune_config=tune.TuneConfig(
        num_samples=8,
        scheduler=pbt,
        metric='accuracy',
        mode='max',
    ),
)
results = tuner.fit()
```

Block-by-Block Explanation

PopulationBasedTraining — The Evolutionary HPO Strategy

Population-Based Training is a hyperparameter optimization algorithm that treats HPO as an evolutionary process rather than a search problem. Rather than running trials with fixed hyperparameters to completion, PBT periodically evaluates all running trials and applies two operations:

- Exploitation — trials in the bottom 25% of performance are terminated and replaced with copies of trials in the top 25%. The new trials start from the top performer's checkpoint, inheriting its learned weights.
- Exploration — after copying a top performer, PBT randomly mutates the hyperparameters (by applying perturbations from hyperparam_mutations). This introduces diversity to prevent all trials from converging to the same local optimum.

PBT is particularly effective for RL training (where hyperparameter sensitivity is very high), fine-tuning large pretrained models (where the hyperparameters that work early in training may differ

from those needed late), and any setting where the optimal hyperparameter schedule changes over the course of training.

perturbation_interval=5 — Evolution Cadence

Every 5 training iterations, PBT evaluates all trials and potentially applies exploit-and-explore operations. A shorter interval makes PBT more aggressive (faster adaptation but higher overhead); a longer interval gives trials more time to stabilize before evaluation. The optimal value depends on how quickly the loss landscape changes and how expensive each training iteration is.

hyperparam_mutations — The Mutation Distribution

hyperparam_mutations defines how hyperparameters are mutated during the exploration phase. For learning rate, a lambda function samples from the log-uniform distribution — so each mutation produces a fresh random learning rate in the valid range. For batch_size, the list [16, 32, 64] means mutations randomly select from these discrete values. PBT applies these mutations only during exploration, not during normal training.

Chapter Summary & Key Takeaways

This chapter has built a comprehensive, end-to-end understanding of distributed AI training — from the fundamental physical constraints that make it necessary, through the three core parallelism strategies, to the engineering challenges of building reliable distributed systems, and finally to the practical implementation of each component using the Ray ecosystem.

Core Conceptual Takeaways

- Single-machine training is permanently insufficient for frontier AI — the memory, compute, and data throughput constraints are physical realities that no single machine can overcome; distributed training is the structural solution
- Data parallelism is the right default starting point — replicate the model, shard the data, synchronize gradients; linear throughput scaling with minimal implementation complexity using PyTorch DDP or Ray Train
- Model parallelism is necessary for models exceeding single-GPU VRAM — pipeline parallelism partitions layers, tensor parallelism partitions weight matrices; both require careful attention to communication overhead and load balancing
- Hybrid 3D parallelism is the architecture of trillion-parameter training — combining tensor, pipeline, and data parallelism at appropriate communication tiers enables training of the world's largest models
- Communication overhead, stragglers, load imbalance, and fault tolerance are the dominant engineering challenges — each requires specific technical mitigations in production distributed training systems

Ray Ecosystem Takeaways

- Ray Core's task-and-actor model provides a Python-native abstraction for any distributed computation pattern — @ray.remote is the single annotation that transforms local Python into distributed Python
- Ray Tune's parallel HPO dramatically accelerates the hyperparameter search process — what takes weeks sequentially completes in hours with parallel trial execution and intelligent early stopping
- Ray Train wraps distributed training complexity in a clean API — changing num_workers scales training from 1 GPU to 1,000 GPUs without modifying training logic
- Ray Serve provides the production serving layer — autoscaling, multi-model composition, and efficient request batching in a unified serving framework
- Anyscale makes enterprise Ray operationally practical — fully managed cluster lifecycle, cost management, and compliance without requiring a dedicated ML infrastructure engineering team

Looking Ahead — Chapter 4

The next chapter explores LLM-specific engineering in depth: Retrieval-Augmented Generation (RAG) architecture, vector database design and operations (Chroma, Pinecone, FAISS), embedding

model selection and fine-tuning, production prompt engineering and versioning, hallucination detection and mitigation frameworks, LLM evaluation methodologies (RAGAS, human evaluation, LLM-as-judge), and cost optimization for large-scale LLM deployments. We also begin implementing a production-grade RAG pipeline from scratch.